AidME - Technical Stack & Budget Optimization Guide

I have thoroughly analyzed the entire codebase (Frontend, Node.js Backend, and Python/ML environment). For your college project budget proposal and maximum performance planning, here is the complete technical stack breakdown and the recommended architectural upgrades.

1. Frontend Stack (User Interface & Client)

Current Technology Used	Version	Purpose	Fast/Modern Replacement for Budget & Speed
React & react-scripts	`^19.0.0` & `^5.0.1`	Core UI library & Webpack bundler (Create React App)	⚡ **Vite (`vite`)**. Create React App is deprecated and significantly slows down large builds and local development. Vite will make your app load and reload almost instantly.
Simple-Peer	`^9.11.1`	WebRTC for Peer-to-Peer (P2P) Video Calling.	⚡ **LiveKit** or **Agora SDK**. `simple-peer` connects directly user-to-user which often drops calls on bad network firewalls and lags. A cloud service like LiveKit uses a central routing server structure (SFU), drastically lowering latency for video calls.
Material-UI (MUI)	`^6.4.11`	React components/Styling system.	Keep it. MUI is enterprise-grade.
Chart.js & Recharts	`^4.5.1` & `^2.15.4`	Displaying medical dashboard graphs.	Keep it. They are optimized and lightweight.
Framer Motion	`^12.16.0`	UI Animations.	Keep it. Excellent for UI aesthetics.
React Leaflet	`^5.0.0`	Maps integration.	Keep it.

2. Backend Stack (Node.js API Server)

Current Technology Used	Version	Purpose	Fast/Modern Replacement for Budget & Speed
Express.js	`^4.21.2`	Core backend framework and REST API logic.	Keep it. Well-tested and handles I/O operations perfectly.
MongoDB & Mongoose	`^6.15.0` & `^8.14.2`	NoSQL Database for patient records.	Keep MongoDB but self-hosting can be slow. Allocate budget for **MongoDB Atlas** (Serverless). Atlas automatically handles index optimization and caching, making reads drastically faster.
OpenAI API	`^4.98.0`	Cloud large language models (GPT).	⚡ **Groq Cloud API**. Groq specializes in Llama models and operates at 800+ tokens per second using custom chips (LPUs). Replacing OpenAI with Groq will result in near-instant AI text responses for 1/10th of the cost.
Google Cloud Speech	`^7.0.0`	Cloud Speech-to-Text inference.	⚡ **Deepgram API**. Deepgram is currently the fastest, most accurate real-time transcription engine designed for developers. It has incredibly low latency (<300ms) compared to Google Cloud.
Socket.io	`^4.8.1`	Real-time WebSockets (Live notifications).	Keep it. Lightweight and effective.
Nodemailer	`^6.10.0`	Sending automated emails/PDF reports.	⚡ **Resend API**. SMTP endpoints via Nodemailer can be slow and easily get marked as spam. Resend is a fast, developer-friendly API that handles mass email delivery rapidly.

3. Machine Learning & Python Stack (AI Integration)

Current Technology Used	Version	Purpose	Fast/Modern Replacement for Budget & Speed
Flask	(Latest)	Python API server serving the AI outputs to Node.js backend.	⚡ **FastAPI**. Replace Flask with FastAPI immediately. FastAPI runs natively on asynchronous operations (ASGI), managing concurrent AI requests much faster than Flask's sync architecture.
Ollama (`llama3.2`)	Local	Generating consultation summaries & prescriptions (RAG).	⚡ Local inference on a CPU/low-end GPU is computationally expensive. For the budget, you can offload this entirely to cloud providers like **Groq API** or **Together.ai**.
WhisperX	`fast_whisper`	Running local transcription.	⚡ Replace with **Deepgram**. Running AI transcription locally requires an expensive NVIDIA proxy. Deepgram processes audio infinitely faster than local CPUs/GPUs could for a fraction of the hardware cost.
ChromaDB	Local	Vector Database for semantic searches in RAG (Medicine Loader).	Keep for prototype. For scale and higher speed, use **Pinecone Serverless** or **Qdrant Cloud** to handle thousands of vector searches unhindered by local memory limits.
YOLOv8 (`ultralytics`)	`yolov8s.pt`	Computer Vision for Cataract and Pneumonia detection.	⚡ **ONNX Runtime Engine.** By exporting your `.pt` YOLO weights to an `.onnx` (Open Neural Network Exchange) format, your model size decreases drastically, allowing for blazing fast execution even on slower computers without heavy PyTorch bloat.
LangChain	(Latest)	Orchestrating RAG AI logic endpoints.	Keep it. Extensively robust utility handling context loaders.

Project Budgeting Recommendations

Make sure to classify your budget components properly. Replacing local, bulky software with serverless API architecture reduces your Capital Expenditure (Buying GPUs/hardware) entirely to minor Operational Expenditure.

1. **AI Processing Cost**: Allocate budget for API credits (**Groq** for LLM generation, **Deepgram** for transcription) instead of requesting budget for an AWS EC2 `p4` GPU instance. This drops your budget to less than $20/month.
2. **Audio/Video Infrastructure**: Include a line item for a Communications Platform as a Service (CPaaS) like **LiveKit Cloud** (or Agora). State that relying purely on `simple-peer` P2P fails hospital firewall policies, and LiveKit guarantees low-latency HIPAA-compliant routing.
3. **Database Infrastructure**: Add **MongoDB Atlas Serverless** cluster to the budget. This is far better than stating you will host your own database, showcasing industry-level redundancy planning.